

Finalización del procesador para Tiny

Procesadores de Lenguaje - GII (UCM)

Grupo 18

Jorge Ortega, Alejandro Tobías

Miguel Amato y Daniela Vidal

Índice

1. Especificación del procesamiento de vinculación.....	3
2. Especificación del procesamiento de comprobación de tipos.....	12
3. Especificación del procesamiento de asignación de espacio.....	21
4. Descripción del repertorio de instrucciones de la máquina-p necesario para soportar la traducción de Tiny a código-p.....	27
5. Especificación del procesamiento de etiquetado.....	30
6. Especificación del procesamiento de generación de código.....	37

1. Especificación del procesamiento de vinculación

var ts

vincula(analiza(Prog)):

ts = crearTS()
vincula(Prog)

vincula(si_decs(LDecs)):

vincula1(LDecs)
vincula2(LDecs)

vincula(no_decs(Decs)):

noop

vincula1(muchas_decs(LDecs, Dec)):

vincula1(LDecs)
vincula1(Dec)

vincula1(una_dec(Dec)):

vincula1(Dec)

vincula1(dec_id(Tipo,id)):

vincula1(Tipo)
if contiene(ts,id):
 error
else
 inserta(ts,id,\$)
end if

vincula1(dec_type(Tipo,id)):

vincula1(Tipo)
if contiene(ts,id):
 error
else
 inserta(ts,id,\$)
end if

vincula1(dec_proc(Id,ParamF,Prog)):

if contiene(ts, Id)
 error
else
 inserta(ts,Id,\$)
end if
inserta(ts, Id,\$)

```
vincula(ParamF)
vincula(Prog)
```

vincula1(tipo_array(Tipo, Ent)):

```
vincula(Tipo)
if esNegativo(Ent)
    error
end if
```

vincula1(tipo_punt(Tipo)):

```
if not Tipo == tipo_id(_) then
    vincula1(Tipo)
```

vincula1(tipo_int()):

```
noop
```

vincula1(tipo_real()):

```
noop
```

vincula1(tipo_string()):

```
noop
```

vincula1(tipo_bool()):

```
noop
```

vincula1(tipo_id(Id)):

```
$.vinculo = vinculoDe(ts,Id)
if not $.vinculo == dec_type(_,_):
    error
end if
```

vincula1(tipo_struct(LStruct)):

```
// PARA SABER SI YA HA DEFINIDO UN CAMPO DEL STRUCT
$.mCampos = new Map()
vincula1(LStruct))
```

vincula1(lista_struct(LStruct, Campo)):

```
vincula 1(LStruct)
vincula1(Campo)
```

vincula1(info_struct(Campo)):

```
vincula1(Campo)
```

vincula1(campo(Tipo,Id)):

```
vincula1(Tipo)
if contiene($.mCampos,id):
    error
else
    inserta($.mCampos, Tipo, Id)
```

```
vincula1(si_parF(LParam)):
    vincula1(LParam)
```

```
vincula1(no_parF()):
    noop
```

```
vincula1(muchos_param(LParam, Param)):
    vincula1(LParam)
    vincula1(Param)
```

```
vincula1(un_param(Param)):
    vincula1(Param)
```

```
vincula1(param_cop(Tipo, Id)):
    vincula1(Tipo)
    if contiene(ts, Id)
        error
    else
        inserta(ts,Id)
```

```
vincula1(param_ref(Tipo, Id)):
    vincula1(Tipo)
    if contiene(ts, Id)
        error
    else
        inserta(ts,Id)
```

```
vincula2(muchas_decs(Decs,Dec)):
    vincula2(Decs)
    vincula2(Dec)
```

```
vincula2(una_dec(Dec)):
    vincula2(Dec)
```

```
vincula2(dec_id(Tipo,Id)):
    vincula2(Tipo)
```

```
vincula2(dec_type(Tipo,Id)):
    vincula2(Tipo)
```

vincula2(dec_proc(Id, ParamF, Prog)):

noop

vincula2(tipo_array(Tipo, Ent)):

vincula2(Tipo)

vincula2(tipo_punt(Tipo)):

```
if Tipo == dec_id(Id)
    if not Tipo.vinculo == dec_type(_,_)
        error
    else
        Tipo.vinculo = vinculoDe(ts,Id)
else
    vincula2(Tipo)
end if
```

vincula2(tipo_int()):

noop

vincula2(tipo_real()):

noop

vincula2(tipo_string()):

noop

vincula2(tipo_bool()):

noop

vincula2(tipo_id(Id)):

noop

vincula2(tipo_struct(LStruct)):

vincula2(LStruct)

vincula2(lista_struct(LStruct, Campo)):

vincula2(LStruct)

vincula2(Campo)

vincula2(info_struct(Campo)):

vincula2(Campo)

vincula2(campo(Tipo,Id)):

vincula2(Tipo)

vincula2(si_parF(LParam)):

vincula2(LParam)

vincula2(no_parF()):

noop

vincula2(muchos_param(LParam, Param)):

vincula2(LParam)

vincula2(Param)

vincula2(un_param(Param)):

vincula2(Param)

vincula2(param_cop(Tipo,Id)):

vincula2(Tipo)

vincula2(param_ref(Tipo,Id)):

vincula2(Tipo)

vincula(si_instr(LInstr)):

vincula(LInstr)

vincula(no_instr()):

noop

vincula(muchas_instr(LInstr,Instr)):

vincula(LInstr)

vincula(Instr)

vincula(una_instr(Instr)):

vincula(Instr)

vincula(instr_eval(Exp)):

vincula(Exp)

vincula(instr_if(Exp,Prog)):

vincula(Exp)

vincula(Prog)

vincula(instr_else(Exp,Prog1,Prog2)):

vincula(Exp)

vincula(Prog1)

vincula(Prog2)

vincula(instr_wh(Exp, Prog)):

vincula(Exp)

vincula(Prog)

vincula(instr_rd(Exp)):

vincula(Exp)

vincula(instr_wr(Exp)):

vincula(Exp)

vincula(instr_nl):

noop

vincula(instr_new(Exp)):

vincula(Exp)

vincula(instr_del(Exp)):

vincula(Exp)

vincula(instr_call(id, ParamR)):

\$.vinculo = vinculoDe(ts, Id)

if not \$.vinculo == p_dec(,_,_) then

error

else

vincula(ParamR)

end if

vincula(instr_comp(Prog)):

vincula(Prog)

vincula(si_param_re(LParamR)):

vincula(LParamR)

vincula(no_param_re()):

noop

vincula(muchos_param_re(LParamR, Exp)):

vincula(LParamR)

vincula(Exp)

vincula(un_param_re(Exp)):

vincula(Exp)

vincula(asig(Opnd0, Opnd1)):

vincula(Opnd0)

vincula(Opnd1)

vincula(mayor(Opnd0, Opnd1)):

vincula(Opnd0)

vincula(Opnd1)

vincula(menor(Opnd0, Opnd1)):

vincula(Opnd0)

vincula(Opnd1)

vincula(mayor_igual(Opnd0, Opnd1)):

vincula(Opnd0)

vincula(Opnd1)

vincula(menor_igual(Opnd0, Opnd1)):

vincula(Opnd0)

vincula(Opnd1)

vincula(igual(Opnd0, Opnd1)):

vincula(Opnd0)

vincula(Opnd1)

vincula(distinto(Opnd0, Opnd1)):

vincula(Opnd0)

vincula(Opnd1)

vincula(suma(Opnd0, Opnd1)):

vincula(Opnd0)

vincula(Opnd1)

vincula(resta(Opnd0, Opnd1)):

vincula(Opnd0)

vincula(Opnd1)

vincula(and(Opnd0, Opnd1)):

vincula(Opnd0)

vincula(Opnd1)

vincula(or(Opnd0, Opnd1)):

vincula(Opnd0)

vincula(Opnd1)

vincula(mul(Opnd0, Opnd1)):

vincula(Opnd0)

vincula(Opnd1)

vincula(mod(Opnd0, Opnd1)):

```
vincula(Opnd0)
vincula(Opnd1)
```

vincula(div(Opnd0, Opnd1)):

```
vincula(Opnd0)
vincula(Opnd1)
```

vincula(menos(Opnd)):

```
vincula(Opnd)
```

vincula(not(Opnd)):

```
vincula(Opnd)
```

vincula(index(Opnd0, Opnd1)):

```
vincula(Opnd0)
vincula(Opnd1)
```

vincula(reg(Opnd, Id)):

```
vincula(Opnd)
```

vincula(indir(Opnd)):

```
vincula(Opnd)
```

vincula(literal_real(R)):

```
noop
```

vincula(literal_ent(Ent)):

```
noop
```

vincula(true()):

```
noop
```

vincula(false()):

```
noop
```

vincula(literal_cadena(S)):

```
noop
```

vincula(id(Id)):

```
$.vinculo = vinculoDe(ts,Id)
if $.vinculo ==  $\perp$  then
    error
end if
```

vincula(null()):

noop

2. Especificación del procesamiento de comprobación de tipos

tipado(analiza(Prog)):

```
tipado(Prog)
$.tipo = Prog.tipo
```

tipado(prog(Decs, InstrOpt)):

```
tipado(Decs)
tipado(InstrOpt)
$.tipo = ambosOK(Decs.tipo, InstrOpt.tipo)
```

tipado(si_decs(LDecs)):

```
tipado(LDecs)
$.tipo = LDecs.tipo
```

tipado(no_decs()):

```
$.tipo = ok
```

tipado(muchas_decs(LDecs, Dec)):

```
tipado(LDecs)
tipado(Dec)
$.tipo = ambosOK(LDecs.tipo, Dec.tipo)
```

tipado(una_dec(Dec)):

```
tipado(Dec)
$.tipo = Dec.tipo
```

tipado(dec_id(Tipo, id)):

```
$.tipo = ok
```

tipado(dec_type(Tipo, id)):

```
$.tipo = ok
```

tipado(dec_proc(id, ParamF, Prog)):

```
tipado(Prog)
$.tipo = Prog.tipo
```

tipado(si_instr(LInstr)):

```
tipado(LInstr)
$.tipo = LInstr.tipo
```

tipado(no_instr()):

```
$.tipo = ok
```

tipado(muchas_instr(LInstr, Instr)):

```
tipado(LInstr)
tipado(Instr)
$.tipo = ambosOK(LInstr.tipo, Instr.tipo)
```

tipado(una_instr(Instr)):

```
tipado(Instr)
$.tipo = Instr.tipo
```

tipado(instr_eval(Exp)):

```
tipado(Exp)
$.tipo = ok
```

tipado(instr_if(Exp, Prog)):

```
tipado(Exp)
tipado(Prog)
if ref!(Exp.tipo) == tipo_bool then
    $.tipo = Prog.tipo
else
    aviso_error(Exp.tipo)
    $.tipo = error
end if
```

tipado(instr_else(Exp, Prog1, Prog2)):

```
tipado(Exp)
tipado(Prog1)
tipado(Prog2)
if ref!(Exp.tipo) == tipo_bool then
    $.tipo = ambosOK(Prog1.tipo, Prog2.tipo)
else
    aviso_error(Exp.tipo)
    $.tipo = error
end if
```

tipado(instr_wh(Exp, Prog)):

```
tipado(Exp)
tipado(Prog)
if ref!(Exp.tipo) == tipo_bool() then
    $.tipo = Prog.tipo
else
    aviso_error(Exp.tipo)
    $.tipo = error
end if
```

tipado(instr_nl()):

```
$.tipo = ok
```

tipado(instr_new(Exp)):

```
tipado(Exp)
if ref!(Exp.tipo) == tipo_punt(T) then
    $.tipo = ok
else
    aviso_error(T)
    $.tipo = error
end if
```

tipado(instr_rd(Exp)):

```
tipado(Exp)
if es_designador(Exp) then
    if (ref!(Exp.tipo) == tipo_int()) or
       (ref!(Exp.tipo) == tipo_real()) or
       (ref!(Exp.tipo) == tipo_string())
        $.tipo = ok
    else
        aviso_error(Exp.tipo)
        $.tipo = error
    end if
else
    error
    $.tipo = error
end if
```

tipado(instr_wr(Exp)):

```
tipado(Exp)
if (ref!(Exp.tipo) == tipo_int()) or (ref!(Exp.tipo) == tipo_real()) or
   (ref!(Exp.tipo) == tipo_string()) or (ref!(Exp.tipo) == tipo_bool())
    $.tipo = ok
else
    aviso_error(Exp.tipo)
    $.tipo = error
end if
```

tipado(instr_del(Exp)):

```
tipado(Exp)
if ref!(Exp.tipo) == tipo_punt(T) then
    $.tipo = ok
else
    aviso_error(T)
    $.tipo = error
end if
```

tipado(instr_call(ld, ParamR)): // TODO

```
let List<ParamF> pf = obtenParamsF($.vinculo.ParamF)
let List<Exp> pc = obtenExps(ParamR)

if len(pf) != len(pc):
    ERROR
for i in range(pf):
    if (pf[i].tipo == tipo_real()) and (pf[i] == param_ref())
        if not es_designador_real(pc[i])
            ERROR
    else if (pf[i] == param_ref())
        if not es_designador(pc[i])
            ERROR
    else
        if not compatible(ref!(pc[i].tipo), pf[i].tipo)
            ERROR
```

tipado(instr_comp(Prog)):

```
tipado(Prog)
$.tipo = Prog.tipo
```

tipado(si_param_re(LParamR)):

```
tipado(LParamR)
$.tipo = LParamR.tipo
```

tipado(no_param_re()):

```
$.tipo = ok
```

tipado(muchos_param_re(LParamR, Exp)):

```
tipado(LParamR)
tipado(Exp)
$.tipo = ambosOK(LParamR.tipo, Exp.tipo)
```

tipado(un_param_re(Exp)):

```
tipado(Exp)
$.tipo = Exp.tipo
```

tipado(asig(Opnd0, Opnd1)):

```
tipado(Opnd0)
tipado(Opnd1)
if es_designador(Opnd0) and compatible(Opnd0.tipo, Opnd1.tipo)
    $.tipo = Opnd0.tipo
else
    aviso_error(Opnd0, Opnd1)
    $.tipo = error
```

tipado(mayor(Opnd0, Opnd1)):

tipado_menor_mayor(Opnd0, Opnd1, \$)

tipado(menor(Opnd0, Opnd1)):

tipado_menor_mayor(Opnd0, Opnd1, \$)

tipado(mayor_igual(Opnd0, Opnd1)):

tipado_menor_mayor(Opnd0, Opnd1, \$)

tipado(menor_igual(Opnd0, Opnd1)):

tipado_menor_mayor(Opnd0, Opnd1, \$)

tipado_menor_mayor(Opnd0, Opnd1, E):

tipado(Opnd0)

tipado(Opnd1)

//Si ref!(0) es int o real y ref!(1) es también int o real, entonces el tipo
resultante es bool

if ref!(Opnd0.tipo) == tipo_int() or ref!(Opnd0.tipo) == tipo_real()
 if ref!(Opnd1.tipo) == tipo_int() or ref!(Opnd1.tipo) == tipo_real()
 E.tipo = tipo_bool()

// Si tanto ref!(0) como ref!(1) son ambos bool, el tipo resultante es también
bool

else if (ref!(Opnd0.tipo) == tipo_bool() and ref!(Opnd1.tipo) == tipo_bool())
 E.tipo = tipo_bool()

// Si tanto ref!(0) como ref!(1) son ambos string, el tipo resultante es bool
else if (ref!(Opnd0.tipo) == tipo_string() and ref!(Opnd1.tipo) == tipo_string())
 E.tipo = tipo_bool()

else

 aviso_error(Opnd0, Opnd1)

 E.tipo = error

tipado(igual(Opnd0, Opnd1)):

tipado_igual_distinto(Opnd0, Opnd1, \$)

tipado(distinto(Opnd0, Opnd1)):

tipado_igual_distinto(Opnd0, Opnd1, \$)

tipado_igual_distinto(Opnd0, Opnd1, \$):

// Si es == o != es lo mismo que menor_mayor con más cosas

tipado(Opnd0)

tipado(Opnd1)

//Si ref!(0) es int o real y ref!(1) es también int o real, entonces el tipo
resultante es bool

```

if ref!(Opnd0.tipo) == tipo_int() or ref!(Opnd0.tipo) == tipo_real()
    if ref!(Opnd1.tipo) == tipo_int() or ref!(Opnd1.tipo) == tipo_real()
        E.tipo = tipo_bool()
    // Si tanto ref!(0) como ref!(1) son ambos bool, el tipo resultante es también
    bool
else if (ref!(Opnd0.tipo) == tipo_bool() and ref!(Opnd1.tipo) == tipo_bool())
    E.tipo = tipo_bool()
// Si tanto ref!(0) como ref!(1) son ambos string, el tipo resultante es bool
else if (ref!(Opnd0.tipo) == tipo_string() and ref!(Opnd1.tipo) == tipo_string())
    E.tipo = tipo_bool()
else if (ref!(Opnd0.tipo) == tipo_punt() and ref!(Opnd1.tipo) == tipo_punt())
    E.tipo = tipo_bool()
else if (ref!(Opnd0.tipo) == tipo_punt() and ref!(Opnd1.tipo) == tipo_null())
    E.tipo = tipo_bool()
else if (ref!(Opnd0.tipo) == tipo_null() and ref!(Opnd1.tipo) == tipo_punt())
    E.tipo = tipo_bool()
else
    aviso_error(Opnd0, Opnd1)
    E.tipo = error

```

tipado(suma(Opnd0, Opnd1)):

```
tipado_aritmetico(Opnd0, Opnd1, $)
```

tipado(resta(Opnd0, Opnd1)):

```
tipado_aritmetico(Opnd0, Opnd1, $)
```

tipado(mul(Opnd0, Opnd1)):

```
tipado_aritmetico(Opnd0, Opnd1, $)
```

tipado(div(Opnd0, Opnd1)):

```
tipado_aritmetico(Opnd0, Opnd1, $)
```

tipado_aritmetico(Opnd0, Opnd1, E):

```
tipado(Opnd0)
tipado(Opnd1)
```

```

if (ref!(Opnd0.tipo) == tipo_int() and ref!(Opnd1.tipo) == tipo_int())
    E.tipo = tipo_int()
else if (ref!(Opnd0.tipo) == tipo_real() and ref!(Opnd1.tipo) == tipo_real())
    E.tipo = tipo_rea()
else if (ref!(Opnd0.tipo) == tipo_int() and ref!(Opnd1.tipo) == tipo_real())
    E.tipo = tipo_rea()
else if (ref!(Opnd0.tipo) == tipo_real() and ref!(Opnd1.tipo) == tipo_int())
    E.tipo = tipo_real()
else

```

```

        aviso_error(Opnd0, Opnd1)
        E.tipo = error
tipado(mod(Opnd0, Opnd1)):
    tipado(Opnd0)
    tipado(Opnd1)

    if (ref!(Opnd0.tipo) == tipo_int() and ref!(Opnd1.tipo) == tipo_int())
        E.tipo = tipo_int()
    else
        aviso_error(Opnd0, Opnd1)
        E.tipo = error

tipado(and(Opnd0, Opnd1)):
    tipado_logico(Opnd0, Opnd1, $)

tipado(or(Opnd0, Opnd1)):
    tipado_logico(Opnd0, Opnd1, $)

tipado_logico(Opnd0, Opnd1, E):
    tipado(Opnd0)
    tipado(Opnd1)

    if (ref!(Opnd0.tipo) == tipo_bool() and ref!(Opnd1.tipo) == tipo_bool())
        E.tipo = tipo_bool()
    else
        aviso_error(Opnd0, Opnd1)
        E.tipo = error

tipado(menos(Opnd)):
    tipado(Opnd)

    if ref!(Opnd.tipo) == tipo_int()
        $.tipo = tipo_int()
    if ref!(Opnd.tipo) == tipo_real()
        $.tipo = tipo_real()
    else
        aviso_error(Opnd0, Opnd1)
        $.tipo = error

tipado(not(Opnd)):
    tipado(Opnd)

    if ref!(Opnd.tipo) == tipo_bool()
        $.tipo = tipo_bool()
    else

```

```

        aviso_error(Opnd0, Opnd1)
        $.tipo = error
tipado(index(Opnd0, Opnd1)):
    tipado(Opnd0)
    tipado(Opnd1)

    if ref!(Opnd0.tipo) == tipo_array(T) and ref!(Opnd1.tipo) == tipo_int()
        $.tipo = T el tipo base del tipo array
    else
        aviso_error(Opnd0, Opnd1)
        $.tipo = error

tipado(reg(Opnd, Id)):
    tipado(Opnd)

    if ref!(Opnd.tipo) == tipo_struct()
        $.tipo = Opnd.tipo.mCampos.tipoDe(id) el tipo del campo (.campo)
    else
        aviso_error(Opnd0, Opnd1)
        $.tipo = error

tipado(indir(Opnd)):
    tipado(Opnd)

    if ref!(Opnd.tipo) == tipo_punt(T)
        $.tipo = T
    else
        aviso_error(Opnd0, Opnd1)
        $.tipo = error

tipado(literal_real(R)):
    $.tipo = tipo_real()

tipado(literal_entero(Ent)):
    $.tipo = tipo_int()

tipado(literal_cadena(S)):
    $.tipo = tipo_string()

tipado(false()):
    $.tipo = tipo_bool()

tipado(true()):
    $.tipo = tipo_bool()

```

tipado(nulo()):

\$.tipo = null

tipado(id(id)):

if \$.vinculo == dec_id(T, l)

\$.tipo = T

else if \$.vinculo = param_cop(T, l) or \$.vinculo = param_ref(T, l)

\$.tipo = T

else

aviso_error()

\$.tipo = error

ambosOK(T0, T1):

if (T0 == error || T1 == error):

return error

else

return ok

3. Especificación del procesamiento de asignación de espacio

```
var dir = 0
var nivel = 0
var max_dir = 0
```

```
asig-espacio(analiza(Prog)):
    asig-espacio(Prog)
```

```
asig-espacio(prog(Decs, InstrOpt)):
    asig-espacio(Decs)
    asig-espacio(InstrOpt)
```

```
asig-espacio(si_decs(LDecs)):
    asig-espacio1(LDecs)
    asig-espacio2(LDecs)
```

```
asig-espacio(no_decs()):
    noop
```

```
asig-espacio1(muchas_decs(LDecs, Dec)):
    asig-espacio1(LDecs)
    asig-espacio1(Dec)
```

```
asig-espacio1(una_dec(Dec)):
    asig-espacio1(Dec)
```

```
asig-espacio1(dec_type(Tipo,id)):
    asig-tam1(Tipo)
```

```
asig-espacio1(dec_id(Tipo, id)):
    asig-tam1(Tipo)
    $.dir = dir
    $.nivel = nivel
    añadir_dir(Tipo.tam)
```

```
asig-espacio1(dec_proc(id, ParamF, Prog)):
    $.dir_ant = dir
    $.max_dir_ant = max_dir
    nivel++
    $.nivel = nivel
    dir = 0
    max_dir = 0
```

```
asig-espacio1(ParamF)
asig-espacio2(ParamF)
asig-espacio(Prog)
$.tam = max_dir
dir = $.dir_ant
max_dir = $.max_dir_ant
nivel--
```

asig-tam1(tipo_array(Tipo,Ent)):

```
asig-tam1(Tipo)
$.tam = Tipo.tam * Ent
```

asig-tam1(tipo_punt(Tipo)):

```
if not (Tipo == ref(id))
    asig-tam1(T)
else
    $.tam = 1
```

asig-tam1(tipo_int()):

```
$.tam = 1
```

asig-tam1(tipo_real()):

```
$.tam = 1
```

asig-tam1(tipo_bool()):

```
$.tam = 1
```

asig-tam1(tipo_string()):

```
$.tam = 1
```

asig-tam1(tipo_id(Id)):

```
let $.vinculo = dec_type(T, id) in
    $.tam = T.tam
```

asig-tam1(tipo_struct(LStruct)):

```
asig-tam1(LStruct)
$.tam = LStruct.tam
```

asig-tam1(lista_struct(LStruct, Campo)):

```
asig-tam1(LStruct)
asig-tam1(Campo, Campos.tam)
$.tam = LStruct.tam + Campo.tam
```

asig-tam1(info_struct(Campo)):

```
asig-tam1(Campo, 0)
```

\$tam = Campo.tam

asig-tam1(campo(Tipo, id), int desp):

asig-tam1(Tipo)
\$.tam = Tipo.tam
\$.desp = desp

asig-espacio1(si_parF(LParams)):

asig-espacio1(LParams)

asig-espacio1(no_parF(LParams)):

noop

asig-espacio1(muchos_param(LParams, ParamF)):

asig-espacio1(LParams)
asig-espacio1(ParamF)

asig-espacio1(un_param(ParamF)):

asig-espacio1(ParamF)

asig-espacio1(param_ref(Tipo,Id)):

asig-tam1(Tipo)
\$.dir = dir
\$.nivel = nivel
añadir_dir(dir)

asig-espacio1(param_cop(Tipo,Id)):

asig-tam1(Tipo)
\$.dir = dir
\$.nivel = nivel
añadir_dir(dir)

asig-espacio2(muchas_decs(Decs, Dec)):

asig-espacio2(Decs)
asig-espacio2(Dec)

asig-espacio2(una_dec(Dec)):

asig-espacio(Dec)

asig-espacio2(dec_type(Tipo, id)):

asig-tam2(Tipo)

asig-espacio2(dec_id(Tipo,Id)):

asig-tam2(Tipo)

asig-espacio2(dec_proc(Id, ParamF, Prog)):

noop

asig-tam2(tipo_array(Tipo,Ent):

asig-tam2(Tipo)

asig-tam2(tipo_punt(Tipo)):

if Tipo == ref(id)

let Tipo.vinculo = dec_type(T, id) in

Tipo.tam = T.tam

else

asig-tam2(T)

asig-tam2(tipo_int()):

noop

asig-tam2(tipo_real()):

noop

asig-tam2(tipo_bool()):

noop

asig-tam2(tipo_string()):

noop

asig-tam2(tipo_id(Id)):

noop

asig-tam2(lista_struct(LStruct, Campo)):

asig-tam2(LStruct)

asig-tam2(Campo)

asig-tam2(tipo_struct(LStruct)):

asig-tam2(LStruct)

asig-tam2(info_struct(Campo)):

asig-tam2(Campo)

asig-tam2(campo(Tipo, Id)):

asig-tam2(Tipo)

asig-espacio2(no_parF()):

noop

asig-espacio2(si_parF(LParam)):

asig-espacio2(LParam)

asig-espacio2(muchos_param(LParam, Param)):

asig-espacio2(LParam)

asig-espacio2(Param)

asig-espacio2(un_param(Param)):

asig-espacio2(Param)

asig-espacio2(param_cop(Tipo, Id)):

asig-tam2(Tipo)

asig-espacio2(param_ref(Tipo, id)):

asig-tam2(Tipo)

asig-espacio(si_instr(LInstr)):

asig-espacio(LInstr)

asig-espacio(no_instr()):

noop

asig-espacio(muchas_instr(LInstr, Instr)):

asig-espacio(LInstr)

asig-espacio(Instr)

asig-espacio(una_instr(Instr)):

asig-espacio(Instr)

asig-espacio(instr_eval(Exp)):

noop

asig-espacio(instr_if(Exp, Prog)):

\$.dir-ent = dir

asig-espacio(Prog)

dir = \$.dir_ant

asig-espacio(instr_else(Exp, Prog1, Prog2)):

\$.dir_ant = dir

asig-espacio(Bloque)

dir = \$.dir_ant

asig-espacio(Bloque)

dir = \$.dir_ant

asig-espacio(instr_wh(Exp, Bloque)):

\$.dir_ant = dir

```
asig-espacio(Bloque)
dir = $.dir_ant
```

```
asig-espacio(instr_rd(Exp)):
  noop
```

```
asig-espacio(instr_wr(Exp)):
  noop
```

```
asig-espacio(instr_nl()):
  noop
```

```
asig-espacio(instr_new(Exp)):
  noop
```

```
asig-espacio(instr_del(Exp)):
  noop
```

```
asig-espacio(instr_call(id, ParamR)):
  noop
```

```
asig-espacio(instr_comp(Prog)):
  $.dir_ant = dir
  asig-espacio(Prog)
  dir = $.dir_ant
```

4. Descripción del repertorio de instrucciones de la máquina-p necesario para soportar la traducción de Tiny a código-p

- **apila-int(n)**: apila el número entero n en la pila de evaluación.
- **apila-real(n)**: apila el número real n en la pila de evaluación
- **apila-string(s)**: apila la cadena de caracteres s en la pila de evaluación.
- **apila-bool(b)**: apila el valor booleano b en la pila de evaluación
- **apila-ind()**: Desapila una dirección d de la pila de evaluación, y apila en dicha pila el contenido de la celda d en la memoria de datos
- **desapila-ind()**: Desapila un valor v y una dirección d de la pila de evaluación (primero v, después d), y actualiza el contenido de la celda d en la memoria de datos a v.
- **mueve(n)**: Desapila dos direcciones d1 y d0 de la pila de evaluación (primero d1, luego d0). Seguidamente copia el contenido de las n celdas consecutivas que comienzan en la dirección d1 a las correspondientes n celdas que comienzan en la dirección d0.
- **copia(n)**: Desapila (i) una dirección d0 ; (ii) una dirección d1 , y, para cada i [0,n), copia el contenido de la celda d0 + i en la celda d1 + i
- **ira(d)**: Salta incondicionalmente a la dirección d
- **irf(d)**: Desapila un valor v de la pila de evaluación. Si es falso salta a d. Si no, continúa en secuencia.
- **irv(d)**: Desapila un valor v de la pila de evaluación. Si es cierto salta a d. Si no, continúa en secuencia.
- **irind**: Desapila una dirección d de la pila de evaluación, y realiza un salto incondicional a dicha dirección.
- **alloc(n)**: Reserva un bloque de n celdas consecutivas en el heap y apila la dirección de comienzo en la pila de evaluación.

- **dealloc(n)**: Desapila una dirección d de la pila de evaluación y libera en el heap el bloque de n celdas consecutivas que comienza en n.
- **activa(n,t,d)** Reserva espacio en el segmento de pila de registros de activación para ejecutar un procedimiento que tiene nivel de anidamiento n y tamaño de datos locales t. Así mismo, almacena en la zona de control de dicho registro d como dirección de retorno. También almacena en dicha zona de control el valor del display de nivel n. Por último, apila en la pila de evaluación la dirección de comienzo de los datos en el registro creado.
- **apilad(n)** Apila en la pila de evaluación el valor del display de nivel n
- **desapilad(n)** Desapila una dirección d de la pila de evaluación en el display de nivel n
- **desactiva(n,t)** Libera el espacio ocupado por el registro de activación actual, restaurando adecuadamente el estado de la máquina. n indica el nivel de anidamiento del procedimiento asociado; t el tamaño de los datos locales. De esta forma, la instrucción: (i) apila en la pila de evaluación la dirección de retorno; (ii) restaura el valor del display de nivel n al antiguo valor guardado en el registro; (iii) decrementa el puntero de pila de registros de activación en el tamaño ocupado por el registro.
- **dup** Consulta el valor v de la cima de la pila de evaluación, y apila de nuevo dicho valor (es decir, duplica la cima de la pila de evaluación)
- **stop** Detiene la máquina
- **mayor()**: Desapila los dos valores de la cima de la pila y apila el resultado de $n1 > n2$.
- **menor()**: Desapila los dos valores de la cima de la pila y apila el resultado de $n1 < n2$.
- **mayor_igual()**: Desapila los dos valores de la cima de la pila y apila el resultado de $n1 \geq n2$.
- **menor_igual()**: Desapila los dos valores de la cima de la pila y apila el resultado de $n1 \leq n2$.
- **igualdad()**: Desapila los dos valores de la cima de la pila y apila el resultado de $n1 == n2$.

- **distinto()**: Desapila los dos valores de la cima de la pila y apila el resultado de $n1 \neq n2$.
- **suma()**: Desapila los 2 valores de la cima de la pila y apila el resultado de $n1 + n2$.
- **resta()**: Desapila los 2 valores de la cima de la pila y apila el resultado de $n1 - n2$.
- **mul()**: Desapila los 2 valores de la cima de la pila y apila el resultado de $n1 * n2$.
- **div()**: Desapila los 2 valores de la cima de la pila y apila el resultado de $n1 / n2$.
- **mod()** : Desapila los 2 valores de la cima de la pila y apila el resultado de $n1 \% n2$.
- **and()** : Desapila los 2 valores de la cima de la pila y apila true si ambos son true o apila false en caso contrario
- **or()**: Desapila los 2 valores de la cima de la pila y apila true si uno de ellos es true o apila false en caso contrario
- **read()**: Coge un valor de la entrada estándar y lo almacena en la dirección de memoria que indica la pila. Después elimina ese valor de la pila.
- **write()**: Escribe el valor de la cima de la pila por salida estándar y lo desapila.
- **menos()**: Desapila el valor de la cima y apila el resultado de cambiar de símbolo este valor
- **not()**: Desapila el valor de la cima y apila el valor contrario (para booleanos)

5. Especificación del procesamiento de etiquetado

```
var etq = 0
var subPend = pilaVacia()
```

etiquetado (prog(Decs,InstrOpt)):

```
$.prim = etq
recolecta_subs(Decs)
etiquetado(Is)
etq++
while !es -vacía (subPend)
  sub = cima(sub_pendientes)
  desapila(sub_pendientes )
  let sub = dec_proc(Id,LParams,Decs,LInstr) in
    sub.prim = etq
    etq++
    recolecta_subs(Decs)
    etiquetado (LInstr)
    etq+=2
    sub.sig = etq
  end let
end while
$.sig = etq
```

etiquetado(si_instr(LInstr)):

```
$.prim = etq
etiquetado(LInstr)
$.sig = etq
```

etiquetado(no_instr(LInstr)):

```
noop
```

etiquetado(muchas_instrs(LInstrs,Exp)):

```
$.prim = etq
etiquetado(LInstrs)
etiquetado(Exp)
$.sig = etq
```

etiquetado(una_instr(Exp)):

```
$.prim = etq
etiquetado(Exp)
$.sig = etq
```

etiquetado(Eval(Exp)):

```
$.prim = etq
```

```
etiquetado(Exp)
etq++
$.sig = etq
```

etiquetado(instr_if(Exp,Bloque)):

```
$.prim = etq
etiquetado(Exp)
etiquetado-acc-val(Exp)
etq++
etiquetado(Bloque)
etq++
$.sig = etq
```

etiquetado(instr_else(Exp,Bloque1, Bloque2)):

```
$.prim = etq
etiquetado(Exp)
etiquetado-acc-val(Exp)
etq++
etiquetado(Bloque1)
etq++
etiquetado(Bloque2)
$.sig = etq
```

etiquetado(instr_wh(Exp, Bloque)):

```
$.prim = etq
etiquetado(Exp)
etiquetado-acc-val(Exp)
etq++
etiquetado(Bloque)
etq++
$.sig = etq
```

etiquetado(instr_rd(Exp)):

```
$.prim = etq
etiquetado(Exp)
etq+=2
$.sig = etq
```

etiquetado(instr_wr(Exp)):

```
$.prim = etq
etiquetado(Exp)
if es_designador(Exp) then
    etq++
etq++
$.sig = etq
```

etiquetado(instr_nl()):

```
$.prim = eq  
etq +=2  
$.sig = etq
```

etiquetado(instr_new(Exp)):

```
$.prim = etq  
etiquetado(Exp)  
etq+=2  
$.sig = etq
```

etiquetado(instr_del(Exp)):

```
$.prim = etq  
etiquetado(Exp)  
etq+=2  
$.sig = etq
```

etiquetado(instr_call(id, ParamR)):

```
$.prim = etq  
etq++  
etiquetado-paso-param($.vinculo, E)  
etq++  
$.sig = etq
```

etiquetado(bq_instr(Bloque)):

```
$.prim = etq  
etiquetado(Bloque)  
$.sig = etq
```

etiquetado-paso-param(si_parF(LParam), si_param_re(LParamR)):

```
etiquetado-paso-param(LParam, LParamR)
```

etiquetado-paso-param(no_parF(),no_param_re()):

```
noop
```

**etiquetado-paso-param(muchos_param(LParam, Param),
muchos_param_re(LParamR, Exp)):**

```
etiquetado-paso-param(LParam, LParamR)  
etq +=3  
etiquetado(Exp)  
etq++
```

etiquetado-paso-param(un_param(Param), un_param_re(Exp)):

```
etq += 3
```

```
etiquetado(Exp)
etq++
```

etiquetado(asig(Exp0, Exp1)):

```
$.prim = etq
etiquetado(Exp0)
etq++
gen-cod(Exp1)
if es_designador(Exp1) then
    etq++
else
    if (ref!(Exp0.tipo) == tipo_real and ref!(Exp1.tipo) == tipo_ent
        etq+=2
    else
        etq++
$.sig = etq
```

etiquetado(mayor(Opnd0, Opnd1)):

```
$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, $)
etq++
$.sig = etq
```

etiquetado(menor(Opnd0, Opnd1)):

```
$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, $)
etq++
$.sig = etq
```

etiquetado(mayor_igual(Opnd0, Opnd1)):

```
$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, $)
etq++
$.sig = etq
```

etiquetado(menor_igual(Opnd0, Opnd1)):

```
$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, $)
etq++
$.sig = etq
```

etiquetado(igual(Opnd0, Opnd1)):

```
$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, $)
etq++
```

\$.sig = etq

etiquetado(distinto(Opnd0, Opnd1)):

\$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, \$)
etq++
\$.sig = etq

etiquetado(suma(Opnd0, Opnd1)):

\$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, \$)
etq++
\$.sig = etq

etiquetado(resta(Opnd0, Opnd1)):

\$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, \$)
etq++
\$.sig = etq

etiquetado(and(Opnd0, Opnd1)):

\$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, \$)
etq++
\$.sig = etq

etiquetado(or(Opnd0, Opnd1)):

\$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, \$)
etq++
\$.sig = etq

etiquetado(mul(Opnd0, Opnd1)):

\$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, \$)
etq++
\$.sig = etq

etiquetado(mod(Opnd0, Opnd1)):

\$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, \$)
etq++
\$.sig = etq

etiquetado(div(Opnd0, Opnd1)):

```
$.prim = etq
etiquetadoOpnds(Opnd0, Opnd1, $)
etq++
$.sig = etq
```

etiquetado(menos(Opnd)):

```
$.prim = etq
etiquetado(Opnd)
if es_designador(Opnd) then
    etq++
etq++
$.sig = etq
```

etiquetado(not(Opnd)):

```
$.prim = etq
etiquetado(Opnd)
if es_designador(Opnd) then
    etq++
etq++
$.sig = etq
```

etiquetado(index(Opnd0, Opnd1)):

```
$.prim = etq
etiquetado(Opnd0)
etiquetado(Opnd1)
etq+=3
$.sig = etq
```

etiquetado(reg(Opnd, Id)):

```
$.prim = etq
etiquetado(Opnd0)
etq+=2
$.sig = etq
```

etiquetado(indireccion(Opnd)):

```
$.prim = etq
etiquetado(Opnd)
etq++
$.sig = etq
```

etiquetado(indir(Opnd)):

```
$.prim = etq
etiquetado(Opnd)
etq++
$.sig = etq
```

etiquetado(literal_real(R)):

```
$.prim = etq
etq++
$.sig = etq
```

etiquetado(literal_ent(Ent)):

```
$.prim = etq
etq++
$.sig = etq
```

etiquetado(literal_cadena(S)):

```
$.prim = etq
etq++
$.sig = etq
```

etiquetado(id(Id)):

```
$.prim = etq
etiquetada-acc-id($.vinculo)
$.sig = etq
```

etiquetado(true()):

```
$.prim = etq
etq++
$.sig = etq
```

etiquetado(false()):

```
$.prim = etq
etq++
$.sig = etq
```

etiquetado(nulo()):

```
$.prim = etq
etq++
$.sig = etq
```

etiquetadoOpnds(Opnd0, Opnd1, Exp):

```
etiquetado(Opnd0)
etiquetado-acc-val(Opnd0)
if Exp == suma(____) or Exp == resta(____) or Exp == mul(____) or Exp == div(____)
    if ref!(Exp.tipo) == tipo_real and ref!(Opnd0.tipo) == tipo_ent
        etq++
etiquetado(Opnd1)
etiquetado-acc-val(Opnd1)
if Exp == suma(____) or Exp == resta(____) or Exp == mul(____) or Exp == div(____)
```

```
if ref!(Exp.tipo) == tipo_real and ref!(Opnd1.tipo) == tipo_ent then  
    etq++
```

etiquetado-acc-val(Exp):

```
if esDesignador(Exp) then  
    etq++  
end if
```

etiquetado-acc-id(Dec_id(T, id)):

```
if $.nivel == 0  
    etq++  
else  
    etiquetado-acc-var($)
```

etiquetado-acc-id(param_cop(T, id)):

```
    etiquetado-acc-var($)
```

etiquetado-acc-id(param_ref(T, id)):

```
    etiquetado-acc-var($)  
    etq++
```

etiquetado-acc-var(V):

```
    etq +=3
```

etiquetado-acc-val(E):

```
    if es_designador(E)  
        etq++
```

recolecta_subs(si_decs(Decs)):

```
    recolecta_subs(Decs)
```

recolecta_subs(no_subs()):

```
    noop
```

recolecta_subs(muchas_decs(LDec, Dec)):

```
    recolecta_subs(LDec)  
    recolecta_subs(Dec)
```

recolecta_subs(una_dec(Dec)):

```
    recolecta_subs(Dec)
```

recolecta_subs(Dec_id(Tipo, id)):

```
    noop
```

recolecta_subs(Dec_tipo(Tipo, id)):

noop

recolecta_subs(Dec_proc(Id,ParamF, Prog)):
apila(subPend, \$)

6. Especificación del procesamiento de generación de código

```
var subPend = pilaVacia()
```

```
gen-cod(analiza(Prog)):  
  gen-cod(Prog)
```

```
gen-cod(prog(Decs,InstrOpt)):  
  recolecta_subs(Decs)  
  gen-co(InstrOpt)  
  emit stop()  
  while !es-vacia(subPend)  
    sub = cima(subPend)  
    desapila(subPend)  
    desapila(subPend)  
    let sub = dec_proc(d,LParams,LDecs,LInstr) in  
      emit desapilaad(sub.nivel)  
      recolecta_subs(LDecs)  
      gen-cod(LInstr)  
      emit desactiva(sub.nivel, sub.tam)  
      emit ir-ind()  
    end let  
  end while
```

```
gen-cod(si_instr(LInstr)):  
  gen-cod(LInstr)
```

```
gen-cod(no_instr()): noop
```

```
gen-cod(muchas_instr(LInstr, Instr)):  
  gen-cod(LInstr)  
  gen-cod(Instr)
```

```
gen-cod(una_instr(Instr)):  
  gen-cod(Instr)
```

```
gen-cod(Eval(Exp)):  
  gen-cod(Exp)  
  emit irA($.prim)
```

```
gen-cod(instr_nl): noop
```

```
gen-cod(instr_rd(Exp)):
```

```
gen-cod(Exp)
emit irA$.prim)
```

gen-cod(instr_write(Exp)):

```
gen-cod(Exp)
emit irA$.prim)
```

gen-cod(instr_new(Exp)):

```
gen-cod(Exp)
let ref! (Exp.tipo) = pointer(Tipo) in
    emit alloc(T.tam)
end
emit desapila-ind()
```

gen-cod(instr_delete(Exp)):

```
gen-cod(Exp)
emit apila-ind()
let ref! (E.tipo) = pointer(T) in
    emit dealloc(T.tam)
end
```

gen-cod(instr_wh(Exp)):

```
gen-cod(Exp)
gen-acc-val(Exp)
emit irF$.sig)
gen-cod(Bloque)
emit irA$.prim)
```

gen-cod(instr_if(Exp, Bloque)):

```
gen-cod(Exp)
gen-acc-val(Exp)
emit irF$.sig)
gen-cod(Bloque)
emit irA$.prim)
```

gen-cod(instr_else(Exp, Bloque1, Bloque2)):

```
gen-cod(Exp)
gen-acc-val(Exp)
emit irF$.sig)
gen-cod(Bloque1)
emit irA$.prim)
gen-cod(Bloque2)
```

gen-cod(bq_instr(Bloque)):

```
gen-cod(Bloque)
```

gen-cod(si_param_re(LParamR)):
gen-cod(LParamR)

gen-cod(no_param_re()): noop

gen-cod(muchos_param_re(LParamR, Exp)):
gen-cod(LParamR)
gen-cod(Exp)

gen-cod(un_param_re(Exp)):
gen-cod(Exp)

gen-cod(asig(Opnd1, Opnd2)):
gen-cod(Opnd1)
gen-cod(Opnd2)
gen-asig(Opnd2)

gen-cod(suma(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, suma)

gen-cod(resta(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, resta)

gen-cod(and(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, and)

gen-cod(or(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, or)

gen-cod(mayor(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, mayor)

gen-cod(menor(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, menor)

gen-cod(mayor_igual(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, mayor_igual)

gen-cod(menor_igual(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, menor_igual)

gen-cod(igual(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, igual)

gen-cod(distintol(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, distinto)

gen-cod(mul(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, mul)

gen-cod(div(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, div)

gen-cod(mod(Opnd1, Opnd2)):
gen-cod-opnds(Opnd1, Opnd2, mod)

gen-cod(literal_entero(N)):
emit apila-int(N)

gen-cod(literal_real(N)):
emit apila-real(N)

gen-cod(literal_cadena(Id)):
emit apila-string(Id)

gen-cod(tipo_array(Tipo, Exp)):
gen-cod(Tipo)
gen-cod(Exp)
gen-acc-val(Exp)
let tipo_array((Tipo, Exp) = ref!(Tipo.tipo) in
 emit apila-int(Tipo.tam)
end let
emit mul
emit suma

gen-cod(tipo_punt(Exp)):
gen-cod(Exp)
emit apila-ind()

gen-cod(campo(Exp, Id)):
gen-cod(Exp)
let tipo_struct(LStruct) = ref!(Exp.tipo) in
 emit apila-int(desplazar(LStruct))
end let
emit suma

gen-asig(Exp):
if esDesignador(ref!(E)) then
 emit copia(E.tipo.tam)

```
else
    emit desapila-ind()
end
```

gen-acc-val(Exp):

```
if esDesignador(ref!(E)) then
    emit apila-ind()
end if
```

gen-cod-opnds(Exp1, Exp2, Exp):

```
gen-cod(Exp1)
gen-acc-val(Exp1)
gen-cod(Exp2)
gen-acc-val(Exp2)
emit Exp
```

desplazar(info_struct(Campo)):

```
emit apila-int(Campo.tipo.tam)
```

desplazar(lista_struct(LStruct, Campo):

```
desplazar(LStruct)
emit apila-int(Campo.tipo.tam)
emit suma
```

recolectaProcs(si_decs(LDecs)):

```
recolectaProcs(LDecs)
```

recolectaProcs(no_decs()): noop

recolectaProcs(muchas_decs(Decs, Dec)):

```
recolectaProcs(Decs)
recolectaProcs(Dec)
```

recolectaProcs(una_dec(Dec)):

```
recolectaProcs(Dec)
```

recolectaProcs(dec_id(Tipo, Id)): noop

recolectaProcs(dec_type(Tipo, Id)): noop

recolectaProcs(dec_proc(Id, ParamF, Prog)):

```
apila(procPend, $)
```